Семинар 2. Pthreads: мьютексы и семафоры

1 Мьютексы

Мьютекс — это примитив синхронизации, которому в `pthreads` соответствует тип данных `pthread_mutex_t`.

Мьютекс инициализируется с помощью функции

```
int pthread_mutex_init(
        pthread_mutex_t*        mutex,    /*out*/
        pthread_mutexattr_t*    attr_p    /*in*/
);
```

Второй аргумент — указатель на атрибуты мьютекса, которые позволяют задать его тип с помощью функции `pthread_mutexattr_settype`:

```
int pthread_mutexattr_settype(
        pthread_mutexattr_t     *attr,    /*out*/
        int                     type      /*in*/
);
```

Некоторые типы мьютексов (второй аргумент `pthread_mutexattr_settype`):
`PTHREAD_MUTEX_NORMAL`
`PTHREAD_MUTEX_ERRORCHECK`
`PTHREAD_MUTEX_RECURSIVE`

Две основные функции для работы с мьютексами:

```
int pthread_mutex_lock(
        pthread_mutex_t* mutex /*in/out*/
);

int pthread_mutex_unlock(
        pthread_mutex_t* mutex /*in/out*/
);
```

Время	Поток 1	Поток 2
0	`pthread_mutex_lock(&m1)`	`pthread_mutex_lock(&m2)`
1	`pthread_mutex_lock(&m2)`	`pthread_mutex_lock(&m1)`

Таблица 1: Дэдлок

Когда происходит выполнение функции `pthread_mutex_lock`, то переданный ей в качестве аргумента мьютекс будет помечен как «закрытый», а его владельцем будет вызвавший функцию поток. Если поток 1 закрыл мьютекс, а поток 2 попытается его еще раз закрыть, то поток 2 будет ожидать в функции `pthread_mutex_lock`, пока поток 1 не откроет его вызовом `pthread_mutex_unlock`. После этого поток 2 закроет мьютекс и продолжит выполнение кода, став новым бладателем мьютекса. Отметим, что **открыть мьютекс может только тот поток, который его закрыл** (который им владеет в данный момент). При попытке открыть мьютекс, закрытый другим потоком, поведение программы не определено.

Дэдлок (deadlock) возникает, когда потоки вынуждены ждать получения мьютекса бесконечно долго. Пример см. в таблице 1.

Освободить ресурсы, выделенные для мьютекса, можно с помощью функции

```
int pthread_mutex_destroy(
        pthread_mutex_t*        mutex     /*in, out*/
);
```

Отличия типов мьютексов друг от друга:

1. `PTHREAD_MUTEX_NORMAL`. Мьютекс по умолчанию. Из соображений скорости работы не осуществляется контроль ошибок. Так, попытка потока залочить уже закрытый им же мьютекс, равно как и попытка открыть закрытый кем-то другим (или открытый) мьютекс, приведет к неопределенному поведению.

2. `PTHREAD_MUTEX_ERRORCHECK`. Мьютекс со встроенной проверкой на ошибки. Если поток попытается закрыть мьютекс, который он же закрывал, то вернется ошибка. Также ошибка будет возвращена, если поток попытается открыть мьютекс, который он не закрывал (или который открыт).

3. `PTHREAD_MUTEX_RECURSIVE`. В отличие от предыдущих двух мьютексов, данный тип допускает повторное закрытие тем же потоком без ошибок или неопределенного поведения. При этом происходит подсчет закрытий и открытий.

Отличие первых двух типов мьютексов между собой демонстрирует листинг 1. Программа выведет сообщения `Operation not permitted`. Если в строке 35 заменить тип мьютекса на `PTHREAD_MUTEX_NORMAL`, то сообщений об ошибках не будет.

Листинг 1: Демонстрация отличий мьютекса с провркой ошибок от обычного

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <errno.h>
6  #define MSG_SIZE 100
7
8  pthread_mutex_t m;
9
10 int thread_count;
11
12 char **messages;
13
14 void *routine(void *rank){
15
16     long my_rank = (long)rank;
17     errno = pthread_mutex_unlock(&m);
18     if (errno != 0)
19         perror("Error");
20
21     return NULL;
22 }
23
24 int main(int argc, char **argv){
25     thread_count = strtol(argv[1], NULL, 10);
26
27     messages = malloc(thread_count * sizeof(char *));
```

```
28
29      pthread_t *thread_handles =
30          malloc(thread_count * sizeof(pthread_t));
31
32      pthread_mutexattr_t attr;
33      pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_ERRORCHECK);
34
35      pthread_mutex_init(&m, &attr);
36      pthread_mutex_lock(&m);
37
38      for (long i = 0; i < thread_count; ++i)
39          pthread_create(&thread_handles[i], NULL, routine, (void *)i);
40
41      for (int i = 0; i < thread_count; ++i)
42          pthread_join(thread_handles[i], NULL);
43
44      free(thread_handles);
45      pthread_mutex_destroy(&m);
46
47  }
```

2 Семафоры

Семафор — это переменная типа `sem_t`, которая представляет собой беззнаковое целое число.

Инициализация семафора

```
int sem_init(
sem_t*       s_p,     /*out*/
int          shared,  /*int*/
int          ini_v    /*int*/
);
```

Второй аргумент — 0, если семафор помещается в область памяти процесса, или 1, если в общую для всех процессов память (последнее нужно для организации межпроцессного взаимодействия). Третий аргумент — начальное значение семафора.

Освобождение ресурсов семафора

```
int sem_destroy(
        sem_t*        s_p,      /*in/out*/
);
```

Две основные команды для работы с семафорами — это поднять семафор, то есть увеличить на единицу (`post`), или опустить, то есть уменьшить на единицу (`wait`).

```
int sem_post(
        sem_t*        s_p,      /*in/out*/
);

int sem_wait(
        sem_t*        s_p,      /*in/out*/
);
```

Если текущее значение семафора единица или выше, то вызов функции `wait` просто уменьшает его на единицу. Если значение семафора равно 0, то вызов функции `wait` приведет к блокировке потока. Поток не покинет функцию `wait`, пока другой поток не выполнит `post` для этого же семафора. Тогда значение семафора станет равный единице, функция `wait` в заблокированном потоке опустит семафор и поток продолжит выполнять дальнейшие инструкции.

Рассмотрим задачу синхронизации типа «производитель — потребитель», простейшая формулировка которой требует от потоков передавать сообщения друг другу по кругу. Корректная реализация алгоритма возможна с использованием семафоров, см. листинг 2.

Листинг 2: Использование симафоров

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define MSG_SIZE 100

sem_t *sems;
int thread_count;
char **messages;
```

```
void *routine(void *rank){

    long my_rank = (long)rank;
    int dest = (my_rank + 1) % thread_count;
    int source = (my_rank + thread_count - 1) % thread_count;
    char *mes = malloc(MSG_SIZE * sizeof(char));
    sprintf(mes, "Hello from %ld to %d", my_rank, dest);
    messages[dest] = mes;
    sem_post(&sems[dest]);
    sem_wait(&sems[my_rank]);
    printf("Thread %ld -> %s\n", my_rank, messages[my_rank]);
        return NULL;
}

int main(int argc, char **argv){

    thread_count = strtol(argv[1], NULL, 10);
    messages = malloc(thread_count * sizeof(char *));

    pthread_t *thread_handles =
        malloc(thread_count * sizeof(pthread_t));

    sems = malloc(thread_count * sizeof(sem_t));

    for (int i = 0; i < thread_count; i++)
        sem_init(&sems[i], 0, 0);

    for (long i = 0; i < thread_count; ++i)
        pthread_create(&thread_handles[i],
            NULL, routine, (void *)i);

    for (int i = 0; i < thread_count; ++i)
        pthread_join(thread_handles[i], NULL);

    free(thread_handles);

    for (int i = 0; i < thread_count; i++){
        sem_destroy(&sems[i]);
```

```
        free(messages[i]);
    }
    free(messages);
    free(sems);

}
```

Бинарный семафор можно использовать в качестве мьютекса. Основное отличие заключается в том, что для семафоров нет понятия владения. Поднять и опустить семафор может любой поток. Что же касается мьютекса, то открыл его может только тот поток, который закрыл (завладел). Небинарный семафор можно использовать, например, для контроля ограниченных ресурсов.